Effectiveness of SBERT-based Sentence Embeddings for Story Point Estimation within the Same Project (Random Split)

Problem statement. PS-5: Investigate the effectiveness of SBERT-based sentence embeddings for story point estimation within the same project using random splitting.

Group 7 - Natural Language Processing course project

#	Name	Roll No.
1	Atluri Venkata Sai Vignesh Chowdary	B230857CS
2	Adimulam Yaswanth Veera Nagesh	B230755CS
3	M V Gowrish	B230673CS
4	Kukkala Sai Dinesh Reddy	B231035CS
5	Munukuntla Rithvik Reddy	B231114CS

GitHub repository: https://github.com/yaswanth230755/group7-ps5-sbert-story-point-estimation

Date generated: 2026-11-05 | **Encoder:** sentence-transformers/all-mpnet-base-v2 | **Seed:** 42 | **Config hash:** 54e10c4846d35aeb

Abstract. We study whether frozen Sentence-BERT (SBERT) embeddings of issue text, fed to a three-layer CNN, predict story points better than simple baselines when training and testing issues come from the same project and are split at random. On 39 projects (62,532 issues) from the TAWOS dataset we use an 80:20 development/test split, five-fold cross-validation on the development part for model selection, and report Mean Absolute Error (MAE) and Median Absolute Error (MdAE) on the held-out test set. Macro-averaged test MAE / MdAE: Mean 2.442 / 1.913; Median 2.165 / 1.372; TF-IDF+Ridge 1.962 / 1.158; SBERT+Ridge 2.017 / 1.227; SBERT+CNN 2.063 / 1.199.

Answer in brief. SBERT+CNN reduced the macro test MAE relative to the median baseline (+4.7%; lower error in 28 of 39 projects). A lexical TF-IDF+Ridge model was better still (macro MAE 1.962 vs 2.063), so SBERT sentence embeddings were useful but not the most effective representation here.

Contents

1. Problem statement and objectives 2

2. Dataset 3

3. Methodology 4

3.1 Pipeline . . . 4

3.2 SBERT sentence embeddings . . . 4

3.3 Prediction component: three-layer CNN . . . 4

3.4 Baselines . . . 4

3.5 Metrics and statistics . . . 5

4. Experimental setup and reproducibility 5

5. Results 5

5.1 Five-fold validation results (development data) . . . 5

5.2 Final held-out test results . . . 7

5.3 Figures . . . 9

5.4 Additional analysis: ranks, win/tie/loss and what explains the gains . . . 12

6. Discussion 13

7. Limitations and threats to validity 14

8. Conclusions 14

9. Demonstration and reproduction 14

References 14

Appendix A. SBERT+CNN held-out test results with 95% confidence intervals 15

Appendix B. Mapping of the assignment's project names to TAWOS names 16

1. Problem statement and objectives

In agile development, issues or user stories are assigned *story points*, a relative measure of effort. Given the text of an issue (title and description), the task is to predict its story point value. Sentence embeddings represent a whole text as one vector; SBERT is trained so that semantically similar texts have similar vectors. **Objective:** investigate how effective SBERT sentence embeddings are for within-project story point estimation under random splitting, measured with MAE and MdAE, and compare against transparent baselines (mean, median, TF-IDF+Ridge, SBERT+Ridge) so that any conclusion about 'effectiveness' is relative to something.

Setting. *Within-project*: training and test issues come from the same project (each project is modelled separately). *Random split*: issues are divided without regard to creation date. Consequently temporal drift is not controlled for (see Limitations).

2. Dataset

We use the TAWOS dataset of open-source agile projects (Tawosi et al., MSR 2022). Only issues with a story point value are usable. Cleaning (see `spsbert/data.py`): non-numeric or non-positive story points removed; text = title + description with code/noformat blocks, Jira macros, HTML tags and user mentions removed and URLs replaced by 'URL'; empty texts removed; duplicate issue keys and exact duplicate texts within a project removed (duplicates would otherwise appear in both training and test data); projects with fewer than 100 usable issues skipped. **Outlier policy:** story points above 100 were removed as data-entry errors (the raw data contain values up to 26,710,114); 141 issues were affected (per project: `results/data_cleaning_report.json`).

Cleaning report: rows raw = 65427, dropped missing or nonpositive story point = 2416, dropped above max story point = 141, dropped empty text = 0, dropped duplicate issue key = 0, dropped duplicate text = 338, rows final = 62532, projects final = 39.

Project names. The assignment (section 3.3) lists the projects by short names, whereas TAWOS uses full names (for example 'XD' is 'Spring XD'). 13 names are identical, 24 differ only by a prefix such as 'Apache', 'Atlassian', 'Hyperledger' or 'Spring', and 2 (Evergreen -> The MongoDB Engineering; C++ driver (Appcelerator) -> Aptana Studio) were matched by elimination because no other name resembles them. The course instructor confirmed by e-mail that the group's mapping may be used. All 39 TAWOS projects are modelled separately, so the mapping changes only the labels in the tables, never a number. The complete mapping is given in Appendix B and in `config/project_aliases.csv`.

Table 1. Story-point distribution per project (mode share = fraction of issues carrying the most frequent value).

Project	Issues	Mean SP	Median	Std	Min	Max	#Distinct	Mode share
Alloy Framework	303	4.13	3.0	2.66	1	21	7	0.39
Apache MXNet	204	6.20	3.0	13.74	0	99	13	0.21
Apache Mesos	3267	3.23	3.0	2.32	0	40	16	0.31
Apache Usergrid	481	2.85	3.0	1.41	1	8	5	0.52
Appcelerator Daemon	241	8.56	5.0	10.27	1	99	12	0.22
Appcelerator Studio	3290	5.62	5.0	3.43	1	55	14	0.39
Aptana Studio	839	8.64	8.0	9.31	1	100	16	0.31
Atlassian Atlassian Software Server	325	4.31	3.0	3.31	1	20	9	0.21
Atlassian Bamboo	514	2.38	2.0	2.18	0	20	10	0.34
Atlassian Clover	382	5.44	2.0	10.97	0	100	12	0.28
Atlassian Confluence Cloud	340	3.09	3.0	2.25	0	21	9	0.35
Atlassian Confluence Server	645	3.22	3.0	1.93	0	21	8	0.42
Atlassian Crowd	213	1.68	1.0	1.32	0	8	9	0.32
Atlassian FishEye	216	1.66	1.0	1.31	0	5	11	0.26
Atlassian Jira Cloud	351	3.96	4.0	1.88	1	8	7	0.52
Atlassian Jira Server	380	4.77	4.0	6.29	1	90	12	0.48
Atlassian Software Cloud	295	4.28	4.0	3.21	1	20	9	0.20
Command-Line Interface	357	3.67	3.0	2.45	1	13	7	0.28
DotNetNuke Platform	2584	2.17	2.0	2.53	0	100	27	0.38
Hyperledger Blockchain Explorer	372	2.96	3.0	1.88	0	13	14	0.42
Hyperledger Fabric	625	2.73	2.0	3.17	1	40	14	0.45
Hyperledger Indy Node	677	3.49	3.0	2.01	1	13	6	0.33
Hyperledger Indy SDK	708	3.89	3.0	2.35	1	13	10	0.34
Hyperledger Sawtooth	941	2.38	2.0	1.31	1	8	5	0.32
Lsstcorp Data management	19655	7.04	3.0	12.54	0	100	288	0.19

Project	Issues	Mean SP	Median	Std	Min	Max	#Distinct	Mode share
Lyrasis Dura Cloud	657	2.10	1.0	2.16	1	21	13	0.53
MongoDB Compass	498	3.36	3.0	1.72	1	8	6	0.42
MongoDB Core Server	743	2.70	2.0	2.68	0	42	11	0.32
Moodle	1524	12.31	5.0	19.36	0	100	24	0.15
Mule	3690	5.02	4.0	4.02	1	32	13	0.22
Mule APIkit	465	3.48	3.0	2.75	1	21	10	0.24
Sonatype Nexus	1803	1.82	1.0	2.08	0	40	15	0.41
Spring DataCass	213	4.47	3.0	8.09	1	100	12	0.37
Spring XD	3538	3.81	3.0	4.42	0	100	24	0.23
The Mongo C++ driver	212	1.37	1.0	0.88	0	8	7	0.66
The Mongo Java driver	238	1.09	1.0	0.99	1	16	4	0.97
The MongoDB Engineering	5064	1.88	2.0	1.12	0	8	8	0.44
The Titanium SDK	4410	5.80	5.0	4.95	0	63	26	0.31
Titanium Mobile Platform	1272	4.52	3.0	4.76	0	40	23	0.32

3. Methodology

3.1 Pipeline

For every project independently: (1) sort issues by key and split 80:20 at random into development and test sets (seed 42); (2) split the development set into five folds; (3) for each method and hyper-parameter setting, train on four folds and validate on the fifth, rotating so every fold validates once; (4) select the setting with the lowest mean validation MAE (ties broken by MdAE) *using development data only*; (5) refit that setting on the whole development set and evaluate once on the untouched test set. The test set is never used for selection, early stopping, scaling or feature fitting. All methods share identical partitions, which allows paired comparisons.

3.2 SBERT sentence embeddings

Each issue text is encoded once with `sentence-transformers/all-mpnet-base-v2` (sentence-transformers; maximum sequence length 384 tokens, longer texts truncated; embeddings L2-normalised). The encoder is frozen and unsupervised w.r.t. the target, so encoding all issues before splitting does not leak labels. Embeddings are cached on disk.

3.3 Prediction component: three-layer CNN

The embedding vector (length D) is treated as a one-channel 1-D signal and passed through three blocks of Conv1d(kernel 5) - BatchNorm - ReLU - avg-pool(2) with 16/32/64 channels, followed by Dropout - Linear(64) - ReLU - Dropout - Linear(1). Targets are transformed with log1p and standardised; inputs are standardised with training statistics; loss = huber; optimiser Adam; batch size 32; at most 100 epochs with early stopping (patience 10) on a 10% hold-out carved from the *training part only*. Predictions are inverse-transformed and clipped to the training range. Hyper-parameter grid (5-fold CV): {'lr': [0.001], 'dropout': [0.1, 0.3, 0.5], 'weight_decay': [0.0001, 0.01]}. Final models are trained with seeds [42, 43, 44]; reported test metrics are means over these seeds.

3.4 Baselines

Mean / Median: predict the training mean / median for all issues. **TF-IDF+Ridge:** word 1-2-gram TF-IDF (max 20000 features, fitted on training data only) with Ridge regression (alpha in [0.1, 1.0, 10.0]). **SBERT+Ridge:** Ridge on the same SBERT vectors (alpha in [0.1, 1.0, 10.0, 100.0]). Ridge models use the same target transform and clipping as the CNN.

Table A. Methods compared and their configuration.

Method	Representation	Model and settings (guideline 3.4)
Mean	none (constant)	training mean
Median	none (constant)	training median
TF-IDF+Ridge	TF-IDF word 1-2-grams	Ridge, alpha in [0.1, 1.0, 10.0], log1p target
SBERT+Ridge	SBERT sentence embeddings	Ridge, alpha in [0.1, 1.0, 10.0, 100.0], log1p target
SBERT+CNN	SBERT sentence embeddings	sentence-transformers/all-mpnet-base-v2; 3x Conv1d [16, 32, 64] k=5 + MLP; huber loss; grid {'lr': [0.001], 'dropout': [0.1, 0.3, 0.5], 'weight_decay': [0.0001, 0.01]}

3.5 Metrics and statistics

MAE = mean(|y - y_hat|); MdAE = median(|y - y_hat|), both in story points (lower is better). Validation results are reported as mean +/- sample standard deviation (ddof = 1) over the five folds and are kept separate from the held-out test results. Test results carry 95% percentile bootstrap confidence intervals (resampling test issues; 1000 resamples) and paired two-sided Wilcoxon signed-rank tests on per-issue absolute errors against the median baseline and TF-IDF+Ridge. Across projects we macro-average (every project counts equally).

4. Experimental setup and reproducibility

Item	Value
Seed / split	42 / 80:20, 5-fold
Encoder	sentence-transformers/all-mpnet-base-v2
Python / PyTorch / sentence-transformers	3.13.15 / 2.11.0+cu128 / 5.7.0
scikit-learn / NumPy / pandas	1.6.1 / 2.1.3 / 2.3.3
Hardware	Tesla T4
Config hash	54e10c4846d35aeb
Independent repeats	The complete run was executed three times on separate days on a Kaggle Tesla T4; all test and validation metrics, split files and predictions were identical.

Seeds, split indices (per-project JSON of issue keys), configuration and software versions are stored in the repository (`results/splits`, `results/run_metadata.json`, `config/`).

5. Results

5.1 Five-fold validation results (development data)

Table 2. Validation metrics of the selected configuration, averaged over projects (fold-wise values for every project: `results/tables/validation_per_fold.csv`).

Method	Val. MAE (macro mean)	Avg. fold std	Val. MdAE (macro mean)	Avg. fold std
Mean	2.484	0.313	1.919	0.232
Median	2.181	0.384	1.354	0.213
TF-IDF+Ridge	1.999	0.360	1.173	0.161
SBERT+Ridge	2.028	0.361	1.180	0.148
SBERT+CNN	2.058	0.364	1.177	0.158

Table 3. SBERT+CNN five-fold validation results per project.

Project	Val. MAE (mean ± std)	Val. MdAE (mean ± std)	Selected hyper-parameters
Alloy Framework	1.715 ± 0.409	1.253 ± 0.189	{'lr': 0.001, 'dropout': 0.1, 'weight_decay': 0.01}
Apache MXNet	4.229 ± 2.887	1.753 ± 0.337	{'lr': 0.001, 'dropout': 0.1, 'weight_decay': 0.01}
Apache Mesos	1.369 ± 0.087	0.920 ± 0.087	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}

Project	Val. MAE (mean ± std)	Val. MdAE (mean ± std)	Selected hyper-parameters
Apache Usergrid	0.957 ± 0.166	0.547 ± 0.070	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.01}
Appcelerator Daemon	5.117 ± 1.614	2.845 ± 0.740	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}
Appcelerator Studio	2.100 ± 0.135	1.689 ± 0.171	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}
Aptana Studio	4.313 ± 0.687	2.456 ± 0.197	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}
Atlassian Atlassian Software Server	2.225 ± 0.373	1.517 ± 0.290	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.01}
Atlassian Bamboo	1.174 ± 0.064	0.755 ± 0.090	{'lr': 0.001, 'dropout': 0.1, 'weight_decay': 0.0001}
Atlassian Clover	4.074 ± 0.831	1.536 ± 0.285	{'lr': 0.001, 'dropout': 0.3, 'weight_decay': 0.0001}
Atlassian Confluence Cloud	1.196 ± 0.124	0.591 ± 0.198	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.01}
Atlassian Confluence Server	1.206 ± 0.074	0.765 ± 0.077	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}
Atlassian Crowd	0.893 ± 0.156	0.666 ± 0.079	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}
Atlassian FishEye	0.958 ± 0.188	0.762 ± 0.144	{'lr': 0.001, 'dropout': 0.3, 'weight_decay': 0.01}
Atlassian Jira Cloud	1.336 ± 0.299	0.981 ± 0.164	{'lr': 0.001, 'dropout': 0.3, 'weight_decay': 0.0001}
Atlassian Jira Server	2.048 ± 0.554	1.184 ± 0.142	{'lr': 0.001, 'dropout': 0.1, 'weight_decay': 0.01}
Atlassian Software Cloud	2.099 ± 0.444	1.445 ± 0.163	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.01}
Command-Line Interface	1.831 ± 0.285	1.780 ± 0.194	{'lr': 0.001, 'dropout': 0.1, 'weight_decay': 0.0001}
DotNetNuke Platform	0.940 ± 0.081	0.583 ± 0.072	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}
Hyperledger Blockchain Explorer	1.124 ± 0.161	0.700 ± 0.119	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.01}
Hyperledger Fabric	1.412 ± 0.192	0.672 ± 0.125	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}
Hyperledger Indy Node	1.415 ± 0.129	1.143 ± 0.100	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}
Hyperledger Indy SDK	1.548 ± 0.127	1.072 ± 0.107	{'lr': 0.001, 'dropout': 0.3, 'weight_decay': 0.01}
Hyperledger Sawtooth	0.937 ± 0.042	0.741 ± 0.140	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}
Lsstcorp Data management	4.670 ± 0.112	1.621 ± 0.062	{'lr': 0.001, 'dropout': 0.3, 'weight_decay': 0.0001}
Lyrasis Dura Cloud	0.972 ± 0.180	0.541 ± 0.076	{'lr': 0.001, 'dropout': 0.3, 'weight_decay': 0.01}
MongoDB Compass	1.288 ± 0.061	1.000 ± 0.067	{'lr': 0.001, 'dropout': 0.1, 'weight_decay': 0.01}
MongoDB Core Server	1.230 ± 0.106	0.788 ± 0.061	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.01}
Moodle	8.954 ± 1.031	3.452 ± 0.667	{'lr': 0.001, 'dropout': 0.1, 'weight_decay': 0.01}
Mule	2.600 ± 0.130	1.785 ± 0.122	{'lr': 0.001, 'dropout': 0.3, 'weight_decay': 0.01}
Mule APIkit	1.656 ± 0.173	1.015 ± 0.172	{'lr': 0.001, 'dropout': 0.1, 'weight_decay': 0.01}
Sonatype Nexus	0.952 ± 0.106	0.577 ± 0.057	{'lr': 0.001, 'dropout': 0.3, 'weight_decay': 0.01}
Spring DataCass	3.138 ± 1.438	1.633 ± 0.148	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.01}
Spring XD	1.842 ± 0.108	1.187 ± 0.048	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}
The Mongo C++ driver	0.508 ± 0.113	0.242 ± 0.117	{'lr': 0.001, 'dropout': 0.5, 'weight_decay': 0.0001}

Project	Val. MAE (mean ± std)	Val. MdAE (mean ± std)	Selected hyper-parameters
The Mongo Java driver	0.127 ± 0.156	0.012 ± 0.009	{'lr': 0.001, 'dropout': 0.1, 'weight_decay': 0.0001}
The MongoDB Engineering	0.749 ± 0.036	0.500 ± 0.010	{'lr': 0.001, 'dropout': 0.1, 'weight_decay': 0.01}
The Titanium SDK	2.876 ± 0.117	1.745 ± 0.068	{'lr': 0.001, 'dropout': 0.3, 'weight_decay': 0.01}
Titanium Mobile Platform	2.494 ± 0.231	1.456 ± 0.187	{'lr': 0.001, 'dropout': 0.3, 'weight_decay': 0.0001}

Table 3b. Hyper-parameters selected by five-fold cross-validation (the CNN learning rate was fixed at 0.001).

Method	Hyper-parameter	Selected values (number of projects)
SBERT+CNN	Dropout	0.1: 10, 0.3: 10, 0.5: 19
SBERT+CNN	Weight decay (L2)	0.0001: 19, 0.01: 20
SBERT+Ridge	Ridge alpha	0.1: 1, 1: 29, 10: 7, 100: 2
TF-IDF+Ridge	Ridge alpha	0.1: 18, 1: 18, 10: 3

5.2 Final held-out test results

Table 4. Test metrics averaged over projects (sorted by MAE). Ties count for the first method.

Method	Test MAE	Test MdAE	Projects with lowest MAE	Projects with lowest MdAE
TF-IDF+Ridge	1.962	1.158	21	13
SBERT+Ridge	2.017	1.227	10	2
SBERT+CNN	2.063	1.199	2	10
Median	2.165	1.372	6	11
Mean	2.442	1.913	0	3

Table 4b. The comparison table required by guideline 3.4 (macro mean over projects). Only the SBERT row belongs to this problem statement (PS-5); the BERT and combined rows are other groups' statements.

Evaluation	Representation	Model configuration	MAE	MdAE
5-fold validation (mean ± std over folds)	BERT word embeddings	not part of PS-5 (assigned to other groups)	-	-
5-fold validation (mean ± std over folds)	SBERT sentence embeddings	sentence-transformers/all-mpnet-base-v2 (frozen, dim=768) + 3-layer CNN channels=[16, 32, 64], kernel=5, avg-pool, loss=huber, Adam; grid={'lr': [0.001], 'dropout': [0.1, 0.3, 0.5], 'weight_decay': [0.0001, 0.01]} selected by 5-fold CV; final seeds=[42, 43, 44]	2.058 ± 0.364	1.177 ± 0.158
5-fold validation (mean ± std over folds)	Combined BERT + SBERT embeddings	not part of PS-5 (assigned to other groups)	-	-
Final held-out test	BERT word embeddings	not part of PS-5 (assigned to other groups)	-	-
Final held-out test	SBERT sentence embeddings	sentence-transformers/all-mpnet-base-v2 (frozen, dim=768) + 3-layer CNN channels=[16, 32, 64], kernel=5, avg-pool, loss=huber, Adam; grid={'lr': [0.001], 'dropout': [0.1, 0.3, 0.5], 'weight_decay': [0.0001, 0.01]} selected by 5-fold CV; final seeds=[42, 43, 44]	2.063	1.199
Final held-out test	Combined BERT + SBERT embeddings	not part of PS-5 (assigned to other groups)	-	-

Table 5. Held-out test MAE / MdAE per project (bold = lowest MAE in the row). 95% confidence intervals and p-values: `results/tables/test_results.csv`.

Project	n test	Mean	Median	TF-IDF+Ridge	SBERT+Ridge	SBERT+CNN
Alloy Framework	61	2.13 / 1.09	1.97 / 2.00	1.96 / 1.33	**1.96 / 1.33**	1.99 / 1.45
Apache MXNet	41	5.78 / 4.29	4.27 / 2.00	**4.21 / 1.34**	4.54 / 1.76	4.32 / 1.48
Apache Mesos	654	1.64 / 1.21	1.55 / 1.00	1.38 / 0.97	**1.33 / 0.93**	1.41 / 0.92
Apache Usergrid	97	1.04 / 0.93	1.03 / 1.00	**0.96 / 0.56**	1.01 / 0.61	1.03 / 0.67

Project	n test	Mean	Median	TF-IDF+Ridge	SBERT+Ridge	SBERT+CNN
Appcelerator Daemon	49	6.60 / 5.42	6.22 / 3.00	**5.33 / 2.79**	5.65 / 3.16	6.09 / 3.26
Appcelerator Studio	658	2.30 / 2.37	2.07 / 2.00	**1.92 / 1.48**	2.00 / 1.51	2.01 / 1.50
Aptana Studio	168	5.23 / 4.42	4.98 / 5.00	**4.74 / 2.81**	4.79 / 2.85	4.81 / 2.75
Atlassian Atlassian Software Server	65	1.97 / 1.40	1.85 / 2.00	1.73 / 1.42	**1.69 / 1.27**	1.84 / 1.53
Atlassian Bamboo	103	1.27 / 0.65	**1.18 / 1.00**	1.19 / 0.86	1.20 / 0.92	1.20 / 0.81
Atlassian Clover	77	4.97 / 3.71	3.14 / 1.00	2.88 / 1.19	**2.85 / 1.31**	3.08 / 1.53
Atlassian Confluence Cloud	68	1.52 / 1.01	1.57 / 0.50	**1.42 / 0.62**	1.45 / 0.60	1.46 / 0.64
Atlassian Confluence Server	129	1.37 / 1.23	**1.24 / 1.00**	1.24 / 0.84	1.25 / 0.90	1.30 / 0.82
Atlassian Crowd	43	0.91 / 0.74	**0.72 / 0.50**	0.76 / 0.62	0.76 / 0.67	0.76 / 0.62
Atlassian FishEye	44	1.15 / 1.00	1.09 / 0.50	**1.05 / 0.77**	1.08 / 0.75	1.09 / 0.72
Atlassian Jira Cloud	71	1.32 / 0.13	**1.24 / 0.00**	1.43 / 0.80	1.47 / 1.08	1.47 / 0.82
Atlassian Jira Server	76	2.07 / 0.79	1.67 / 0.00	1.77 / 0.52	1.65 / 1.16	**1.64 / 1.03**
Atlassian Software Cloud	59	1.77 / 1.33	1.86 / 1.00	1.77 / 1.56	1.79 / 1.37	**1.74 / 1.31**
Command-Line Interface	72	1.86 / 1.34	1.75 / 2.00	**1.66 / 1.30**	1.79 / 1.48	1.87 / 1.59
DotNetNuke Platform	517	0.96 / 0.82	0.87 / 1.00	**0.77 / 0.53**	0.81 / 0.53	0.86 / 0.61
Hyperledger Blockchain Explorer	75	1.41 / 0.93	1.40 / 1.00	**1.30 / 0.87**	1.33 / 1.11	1.35 / 0.98
Hyperledger Fabric	125	1.80 / 1.71	1.65 / 1.00	1.27 / 0.80	**1.27 / 0.74**	1.40 / 0.78
Hyperledger Indy Node	136	1.56 / 1.50	1.48 / 1.50	**1.28 / 1.05**	1.35 / 1.18	1.37 / 1.10
Hyperledger Indy SDK	142	1.70 / 1.07	1.46 / 1.00	1.31 / 1.08	**1.28 / 0.90**	1.32 / 0.84
Hyperledger Sawtooth	189	0.99 / 0.62	0.92 / 1.00	**0.77 / 0.70**	0.81 / 0.63	0.84 / 0.66
Lsstcorp Data management	3931	6.88 / 5.04	5.62 / 2.00	**4.31 / 1.64**	4.65 / 1.81	4.56 / 1.59
Lyrasis Dura Cloud	132	1.38 / 1.03	1.40 / 1.00	1.20 / 0.63	**1.17 / 0.64**	1.21 / 0.61
MongoDB Compass	100	1.33 / 1.34	1.19 / 1.00	**1.19 / 1.00**	1.20 / 0.99	1.27 / 0.96
MongoDB Core Server	149	1.78 / 1.65	1.62 / 1.00	**1.42 / 0.83**	1.47 / 0.97	1.49 / 0.87
Moodle	305	10.74 / 7.71	9.31 / 3.00	**7.85 / 3.37**	8.25 / 3.52	8.76 / 3.70
Mule	738	2.86 / 2.95	2.71 / 2.00	**2.35 / 1.70**	2.37 / 1.75	2.45 / 1.79
Mule APIkit	93	1.77 / 1.56	1.57 / 2.00	1.62 / 1.40	**1.41 / 1.20**	1.47 / 1.16
Sonatype Nexus	361	1.14 / 0.81	1.02 / 0.50	0.97 / 0.61	**0.97 / 0.61**	0.98 / 0.58
Spring DataCass	43	2.94 / 3.51	2.84 / 2.00	**2.57 / 1.84**	2.87 / 2.12	2.77 / 1.88
Spring XD	708	2.51 / 1.72	2.39 / 2.00	**2.09 / 1.20**	2.10 / 1.22	2.14 / 1.23
The Mongo C++ driver	43	0.59 / 0.38	**0.41 / 0.00**	0.48 / 0.31	0.45 / 0.32	0.46 / 0.26
The Mongo Java driver	48	0.12 / 0.12	**0.00 / 0.00**	0.04 / 0.04	0.05 / 0.04	0.01 / 0.01
The MongoDB Engineering	1013	0.79 / 0.87	0.77 / 1.00	0.73 / 0.54	**0.73 / 0.52**	0.77 / 0.50
The Titanium SDK	882	3.43 / 2.77	3.13 / 2.00	**2.90 / 1.81**	2.95 / 1.88	3.00 / 1.81
Titanium Mobile Platform	255	3.66 / 3.47	3.30 / 2.00	**2.71 / 1.46**	2.93 / 1.54	2.90 / 1.37

5.3 Figures

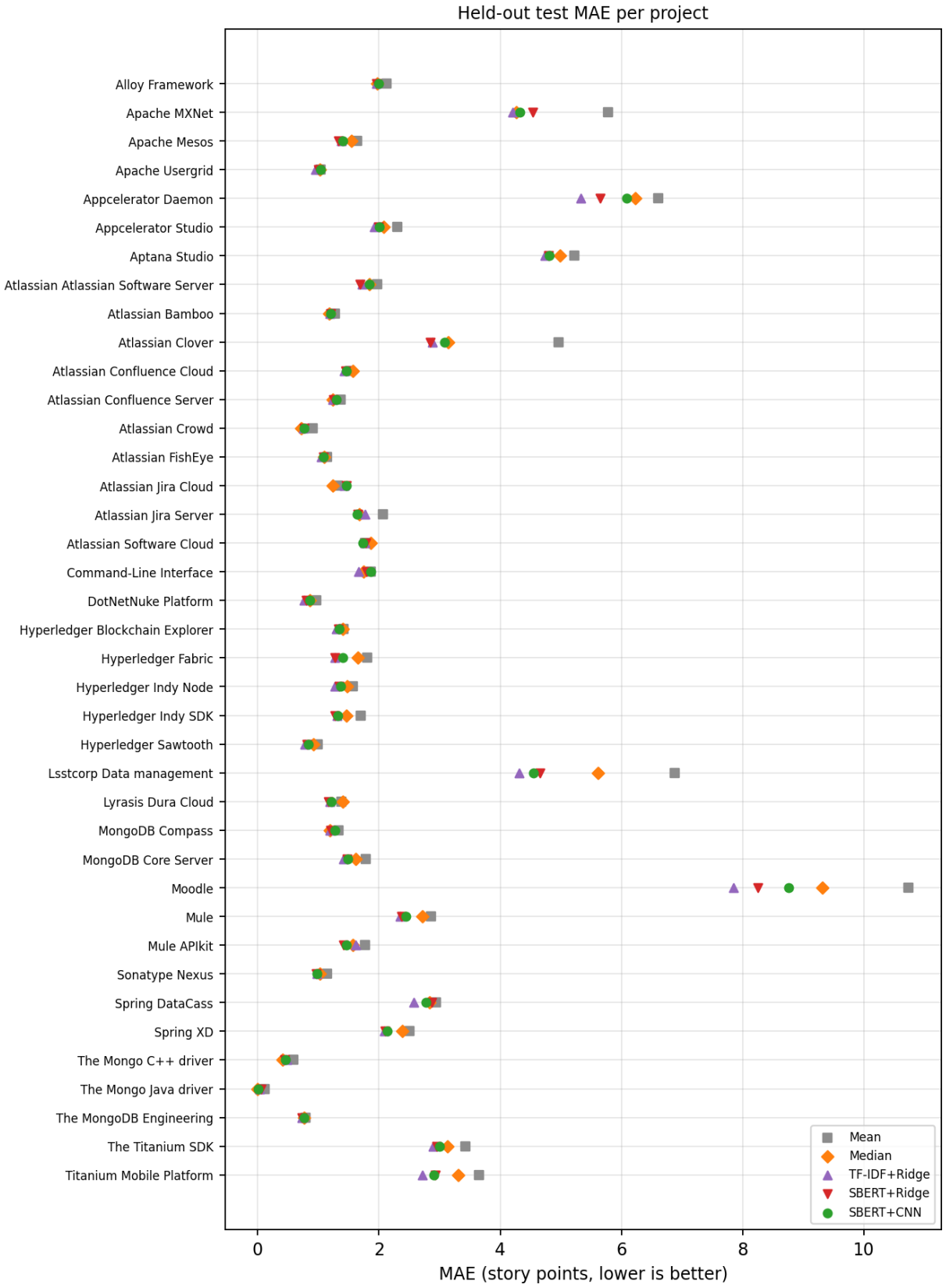


Figure 1. Test MAE per project.



Figure 2. Test MdAE per project.



Figure 3. Relative MAE change versus the median baseline.

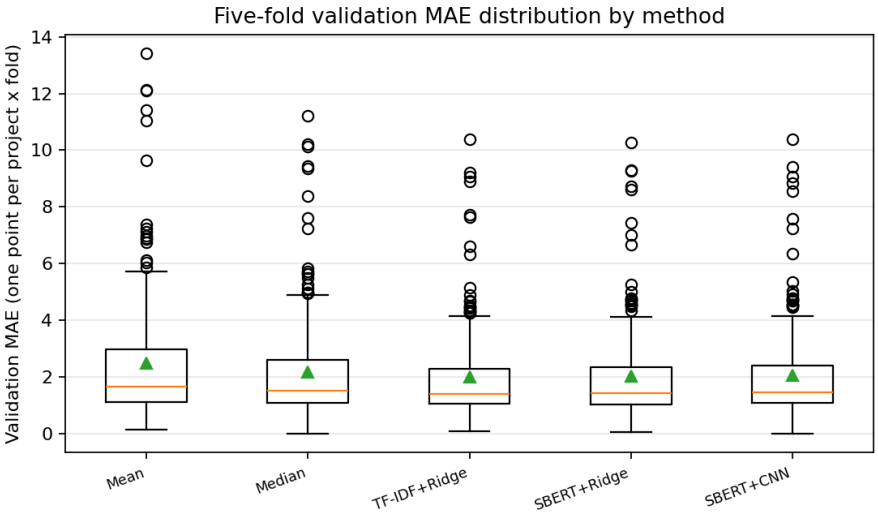


Figure 4. Distribution of fold-wise validation MAE.

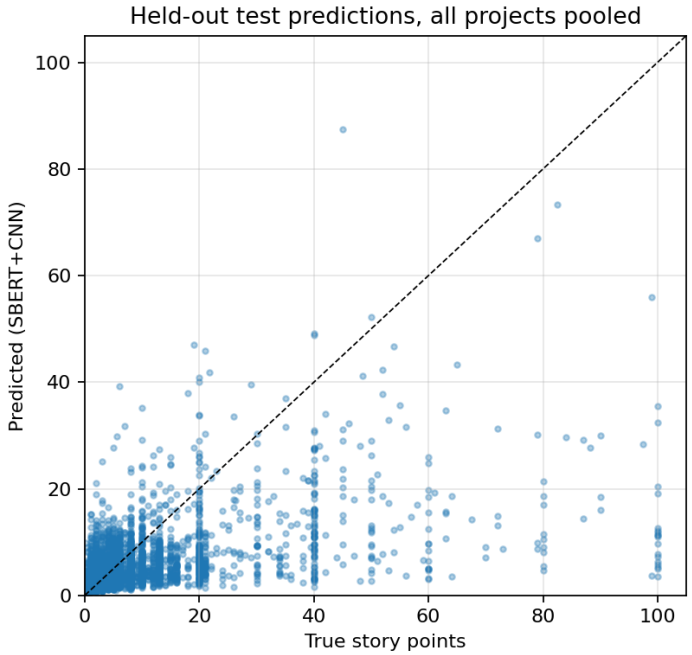


Figure 5. SBERT+CNN predictions versus true values on the test sets (all projects).

5.4 Additional analysis: ranks, win/tie/loss and what explains the gains

Table 6. Average rank over projects.

Method	Mean rank by test MAE (1 = best)
TF-IDF+Ridge	1.67
SBERT+Ridge	2.23
SBERT+CNN	2.97
Median	3.33
Mean	4.79

A Friedman test over the 39 projects rejects equal performance of the methods (chi-square = 88.96, p = 2.2e-18).

Table 7. Per-project comparison of SBERT+CNN with each other method (lower MAE = win).

SBERT+CNN against	Wins	Ties	Losses	Wilcoxon p (per-project MAE)
Mean	37	0	2	<0.0001
Median	28	0	11	0.0006
TF-IDF+Ridge	5	0	34	<0.0001
SBERT+Ridge	9	0	30	0.0016

Spearman correlation between a project's property and SBERT+CNN's MAE improvement over the median baseline: project size (issues): rho = +0.55 ($p < 0.001$); mode share of the most frequent story point: rho = -0.28 ($p = 0.090$); number of distinct story-point values: rho = +0.52 ($p < 0.001$); story-point standard deviation: rho = +0.34 ($p = 0.039$). These correlations describe the 39 projects and are not causal evidence.

6. Discussion

All statements below are computed from the measured result tables (results/tables) and describe these experiments only.

Averaged over 39 projects (each project weighted equally), the lowest held-out test MAE was obtained by **TF-IDF+Ridge** (1.962) and the lowest MdAE by **TF-IDF+Ridge** (1.158).

Among the SBERT-based models, SBERT+Ridge has the lowest MAE (2.017) and SBERT+CNN the lowest MdAE (1.199); this is the answer for the assigned (SBERT sentence-embedding) representation.

SBERT+CNN improves on the trivial median-of-training predictor in macro-averaged test MAE (2.063 vs 2.165, +4.7%) and has MdAE 1.199 vs 1.372.

Per project, SBERT+CNN had lower test MAE than the median baseline in 28 of 39 projects.

A Wilcoxon signed-rank test across projects on test MAE (SBERT+CNN vs median) gives $p = 0.0006$ ($n = 39$); the 39 projects are not a random sample of all software projects, so p-values are descriptive of these projects rather than confirmatory for software projects in general.

On individual projects, the paired per-issue Wilcoxon test against the median baseline was significant at 0.05 (uncorrected) for 15 projects, of which SBERT+CNN was better in 12; after Holm correction across projects it remained significant for 8.

A linear Ridge head on the same SBERT vectors matched or beat the CNN in macro test MAE (SBERT+Ridge 2.017 vs SBERT+CNN 2.063); so on average the CNN's extra non-linear capacity did not pay off at these sample sizes (most projects have only a few hundred training issues).

Comparing representations with the same Ridge head, SBERT embeddings did not outperform lexical TF-IDF features in macro test MAE (2.017 vs 1.962).

By average rank over projects the best method is TF-IDF+Ridge (1.67); a Friedman test indicates that the methods differ ($p = 2.2e-18$).

SBERT+CNN's advantage over the median baseline grows with project size (Spearman rho = +0.55, $p < 0.001$), consistent with a neural model needing more training data.

Possible explanations (hypotheses suggested by the data, not tested here): (i) story points are discrete; in these data the most frequent value covers on average 36% of a project's issues (see 'mode share' in Table 1), and the higher this share, the stronger a constant predictor is for MdAE; (ii) estimates reflect team-specific conventions and context (sprint capacity, assignee) that are not in the text; (iii) the frozen SBERT encoder is trained for semantic similarity, not effort; (iv) small projects give high-variance fits and wide confidence intervals.

Which representation achieves the lowest MAE and MdAE? See the first sentence of this section and Table 4. This project covers the SBERT arm only; the BERT-word and combined arms of the overall study are other groups' projects, and the stored split files (issue keys) allow a paired comparison with them on identical partitions.

7. Limitations and threats to validity

• **Random splitting** lets the model see issues created after the test issues (temporal leakage / optimistic estimates relative to real deployment); this is by design for PS-5 but limits external validity.

• **Single random 80:20 split per project.** Test sets of small projects are small, so metrics are noisy; bootstrap intervals quantify only test-sampling variability, not split variability.

• **Near-duplicates.** Only exact duplicate texts are removed; templated or cloned issues can still appear on both sides of a split and flatter any text model.

• **Outlier policy.** Removing story points above the planning-poker maximum is a modelling decision that the assignment does not specify; it changes the eligible data and the absolute MAE values, and results without the cap would differ (especially for projects with extreme data-entry errors).

• **Label noise and context.** Story points are relative, team-specific and sometimes assigned after the fact; effort drivers outside the text are invisible to the model.

• **Frozen encoder and truncation.** Texts beyond the encoder's maximum length are truncated; the encoder was not fine-tuned for effort.

• **Multiple comparisons.** Per-project p-values are reported both uncorrected and Holm-corrected; cross-project tests have few observations.

• **Hyper-parameter search is small** and the CNN treats embedding dimensions as ordered positions, which has no semantic justification; results for the CNN may differ with other architectures.

• **Dataset heterogeneity.** Projects differ in size, scale and estimation practice, so macro averages hide large differences.

8. Conclusions

On the evaluated projects, the best macro-averaged test MAE was achieved by TF-IDF+Ridge (1.962) and the best MdAE by TF-IDF+Ridge (1.158). SBERT+CNN changed macro test MAE by +4.7% relative to the median baseline. This supports a modest benefit of SBERT-based estimation over a trivial baseline in the within-project, random-split setting, although the lexical TF-IDF+Ridge baseline was better still (macro MAE 1.962 vs 2.063), so frozen SBERT embeddings were not the most effective representation here. These conclusions are subject to the limitations above.

9. Demonstration and reproduction

The implementation can be demonstrated and the principal results reproduced in three ways: (i) run the notebook `notebooks/Group7_PS5_SBERT_StoryPoints.ipynb` (Kaggle with a T4 GPU, about 80 minutes; Section 13 shows a live prediction and a recomputation check); (ii) use the command line below; (iii) inspect the stored results and the strict audit `python -m spsbert check`. Every number in this report is read from `results/`.

```
git clone https://github.com/yaswanth230755/group7-ps5-sbert-story-point-estimation.git
cd group7-ps5-sbert-story-point-estimation
pip install -r requirements.txt && pip install -e .
python -m spsbert sql2csv path/to/TAWOS.sql --out data/tawos_issues.csv
python -m spsbert run --config config/default.yaml
python -m spsbert verify --project "Apache MXNet"
python -m spsbert predict --project "Apache MXNet" --text "Fix typo in the label of the save
button"
python -m spsbert check
```

References

[1] N. Reimers and I. Gurevych, Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, EMNLP-IJCNLP 2019.

[2] V. Tawosi, A. Al-Subaihin, R. Moussa, F. Sarro, A Versatile Dataset of Agile Open Source Software Projects, MSR 2022 (TAWOS; data DOI 10.5522/04/21308124).

[3] M. Choetkiertikul et al., A deep learning model for estimating story points (Deep-SE), IEEE Transactions on Software Engineering.

[4] K. Song et al., MPNet: Masked and Permuted Pre-training for Language Understanding, NeurIPS 2020 (backbone of all-mpnet-base-v2).

[5] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1(6), 1945.

Appendix A. SBERT+CNN held-out test results with 95% confidence intervals

Table A1. Intervals come from 1,000 bootstrap resamples of the test issues; the last column is the Holm-corrected Wilcoxon p-value against the median baseline.

Project	n test	MAE [95% CI]	MdAE [95% CI]	Holm p vs median
Alloy Framework	61	1.99 [1.53, 2.55]	1.45 [1.04, 1.84]	1.000
Apache MXNet	41	4.32 [1.61, 9.30]	1.48 [1.06, 2.08]	1.000
Apache Mesos	654	1.41 [1.29, 1.53]	0.92 [0.85, 0.99]	<0.001
Apache Usergrid	97	1.03 [0.84, 1.25]	0.67 [0.37, 0.93]	1.000
Appcelerator Daemon	49	6.09 [4.20, 8.58]	3.26 [2.86, 4.00]	1.000
Appcelerator Studio	658	2.01 [1.88, 2.16]	1.50 [1.46, 1.60]	1.000
Aptana Studio	168	4.81 [3.52, 6.29]	2.75 [2.29, 3.36]	1.000
Atlassian Atlassian Software Server	65	1.84 [1.47, 2.24]	1.53 [1.30, 1.73]	1.000
Atlassian Bamboo	103	1.20 [0.90, 1.61]	0.81 [0.66, 0.97]	1.000
Atlassian Clover	77	3.08 [1.64, 5.47]	1.53 [1.30, 1.82]	1.000
Atlassian Confluence Cloud	68	1.46 [0.93, 2.14]	0.64 [0.51, 0.76]	0.060
Atlassian Confluence Server	129	1.30 [1.02, 1.69]	0.82 [0.67, 1.04]	1.000
Atlassian Crowd	43	0.76 [0.56, 1.07]	0.62 [0.48, 0.74]	1.000
Atlassian FishEye	44	1.09 [0.81, 1.40]	0.72 [0.62, 0.86]	1.000
Atlassian Jira Cloud	71	1.47 [1.13, 1.77]	0.82 [0.59, 1.26]	0.001
Atlassian Jira Server	76	1.64 [1.32, 2.01]	1.03 [0.78, 1.53]	0.702
Atlassian Software Cloud	59	1.74 [1.33, 2.20]	1.31 [1.07, 1.63]	1.000
Command-Line Interface	72	1.87 [1.53, 2.26]	1.59 [1.30, 1.96]	1.000
DotNetNuke Platform	517	0.86 [0.77, 0.96]	0.61 [0.58, 0.63]	1.000
Hyperledger Blockchain Explorer	75	1.35 [1.07, 1.67]	0.98 [0.66, 1.33]	1.000
Hyperledger Fabric	125	1.40 [1.09, 1.81]	0.78 [0.64, 0.97]	<0.001
Hyperledger Indy Node	136	1.37 [1.17, 1.58]	1.10 [0.89, 1.29]	1.000
Hyperledger Indy SDK	142	1.32 [1.12, 1.53]	0.84 [0.69, 1.04]	0.851
Hyperledger Sawtooth	189	0.84 [0.75, 0.93]	0.66 [0.57, 0.74]	0.431
Lsstcorp Data management	3931	4.56 [4.28, 4.85]	1.59 [1.53, 1.66]	<0.001
Lyrasis Dura Cloud	132	1.21 [0.91, 1.55]	0.61 [0.52, 0.70]	1.000
MongoDB Compass	100	1.27 [1.08, 1.49]	0.96 [0.75, 1.31]	0.645
MongoDB Core Server	149	1.49 [1.18, 1.84]	0.87 [0.77, 0.99]	1.000
Moodle	305	8.76 [7.06, 10.68]	3.70 [3.28, 4.37]	1.000
Mule	738	2.45 [2.27, 2.64]	1.79 [1.65, 1.93]	<0.001
Mule APIkit	93	1.47 [1.23, 1.72]	1.16 [0.92, 1.41]	1.000
Sonatype Nexus	361	0.98 [0.80, 1.25]	0.58 [0.51, 0.64]	1.000
Spring DataCass	43	2.77 [2.08, 3.55]	1.88 [1.62, 2.46]	1.000

Project	n test	MAE [95% CI]	MdAE [95% CI]	Holm p vs median
Spring XD	708	2.14 [1.84, 2.56]	1.23 [1.12, 1.35]	<0.001
The Mongo C++ driver	43	0.46 [0.33, 0.62]	0.26 [0.19, 0.47]	1.000
The Mongo Java driver	48	0.01 [0.01, 0.01]	0.01 [0.01, 0.01]	<0.001
The MongoDB Engineering	1013	0.77 [0.72, 0.81]	0.50 [0.48, 0.52]	1.000
The Titanium SDK	882	3.00 [2.77, 3.25]	1.81 [1.67, 1.97]	0.141
Titanium Mobile Platform	255	2.90 [2.38, 3.54]	1.37 [1.18, 1.59]	<0.001

Appendix B. Mapping of the assignment's project names to TAWOS names

Table B1. 26 of the 39 names differ; the other 13 are identical.

Name in the assignment (section 3.3)	Name in TAWOS	Basis
Atlassian Software Server	Atlassian Atlassian Software Server	name pattern
Bamboo	Atlassian Bamboo	name pattern
Blockchain Explorer	Hyperledger Blockchain Explorer	name pattern
C++ driver (Appcelerator)	Aptana Studio	by elimination
C++ driver (MongoDB)	The Mongo C++ driver	name pattern
Clover	Atlassian Clover	name pattern
Compass	MongoDB Compass	name pattern
Confluence Cloud	Atlassian Confluence Cloud	name pattern
Confluence Server	Atlassian Confluence Server	name pattern
Crowd	Atlassian Crowd	name pattern
DataCass	Spring DataCass	name pattern
Evergreen	The MongoDB Engineering	by elimination
Fabric	Hyperledger Fabric	name pattern
FishEye	Atlassian FishEye	name pattern
Indy Node	Hyperledger Indy Node	name pattern
Indy SDK	Hyperledger Indy SDK	name pattern
Java driver	The Mongo Java driver	name pattern
Jira Cloud	Atlassian Jira Cloud	name pattern
Jira Server	Atlassian Jira Server	name pattern
MESOS	Apache Mesos	name pattern
MXNet	Apache MXNet	name pattern
Nexus	Sonatype Nexus	name pattern
Sawtooth	Hyperledger Sawtooth	name pattern
Software Cloud	Atlassian Software Cloud	name pattern
Usergrid	Apache Usergrid	name pattern
XD	Spring XD	name pattern